ПМ3 Разработка модулей ПО.

РО 3.1 Понимать и применять принципы объектно-ориентированного и асинхронного программирования.

Тема 2. Асинхронно программирование.

Лекция 10. Основы асинхронности и стек вызовов.

Цели занятия:

- Понять, как работает однопоточная модель JavaScript.

- Понять различие между синхронным и асинхронным кодом.

- Узнать реальные области применения асинхронности (сетевые запросы, работа с таймерами, событиями).

- Понять, что такое Call Stack (стек вызовов), как работает Event Loop, чем отличаются macro- и microtasks

Учебные вопросы:

1. Что такое Event Loop и зачем он нужен
2. Как Event Loop управляет синхронным и асинхронным кодом?
3. Macrotasks — что это и примеры
4. Microtasks — что это и примеры
5. queueMicrotask — добавление микрозадачи вручную

1. Что такое Event Loop и зачем он нужен.

Проблема: JavaScript однопоточный

Мы уже знаем — JavaScript выполняет код в одном потоке. Это означает, что в один момент времени может выполняться только одна операция. Пока выполняется текущая задача — всё остальное стоит и ждёт.

Это порождает вопрос: как тогда работают таймеры, запросы к серверу, события мыши? Ведь они могут сработать в любой момент — пока выполняется другой код. Кто следит за ними и решает когда их запустить?

Ответ — Event Loop.

Что такое Event Loop?

Event Loop (цикл событий) — это механизм, который постоянно следит за двумя вещами:

- пуст ли Call Stack (стек вызовов)
- есть ли задачи в очереди ожидания

Как только стек освобождается — Event Loop берёт следующую задачу из очереди и отправляет её на выполнение.

Проще говоря: Event Loop — это диспетчер, который решает что выполнять следующим.

Участники процесса

Чтобы понять Event Loop, нужно знать всех участников:

- **Call Stack** — стек вызовов. Здесь выполняется текущий код. Функции заходят в стек и выходят из него по мере выполнения. Пока стек не пуст — Event Loop ждёт.
- **Web API** (в браузере) / Node.js API (в Node) — внешняя среда, которая берёт на себя «долгие» операции. Когда мы пишем setTimeout — JavaScript передаёт таймер браузеру и продолжает выполнять код дальше. Браузер следит за таймером сам.
- **Очередь задач** — когда браузер заканчивает свою работу (таймер истёк, пришёл ответ от сервера, пользователь кликнул) — колбэк отправляется в очередь и ждёт своей очереди.
- **Event Loop** — смотрит на стек и очередь. Как только стек пуст — перемещает задачу из очереди в стек.

Пошаговый разбор — синхронный код и queueMicrotask

Посмотрим как Event Loop работает на конкретном примере. queueMicrotask — это встроенная функция, которая добавляет задачу в очередь микрозадач. Она выполнится асинхронно — но раньше чем любой таймер:

```
console.log("старт");

queueMicrotask(() => {
    console.log("асинхронно");
});

console.log("конец");
```

Разберём шаг за шагом:

- Шаг 1: console.log("старт") попадает в стек, выполняется, выводит "старт", выходит из стека.
- Шаг 2: queueMicrotask(...) попадает в стек. JavaScript регистрирует колбэк в очереди микрозадач и сразу выходит из стека — колбэк не выполняется прямо сейчас.
- Шаг 3: console.log("конец") попадает в стек, выполняется, выводит "конец", выходит из стека.
- Шаг 4: Стек пуст. Event Loop проверяет очередь микрозадач — там есть колбэк. Перемещает его в стек.
- Шаг 5: Колбэк выполняется, выводит "асинхронно", выходит из стека.

Вывод:

```
PS C:\Users\user\Documents\oop> node index.js
старт
конец
асинхронно
```

Ключевой момент — queueMicrotask стоит в коде раньше второго console.log, но выполнился позже. Потому что он не попал в стек сразу — он встал в очередь и ждал пока стек освободится.

Реальные сценарии где нужен Event Loop

Event Loop — это то, что делает JavaScript практически применимым несмотря на однопоточность. Без него JavaScript замирал бы при каждой долгой операции.

- Таймеры. Когда нужно выполнить код через определённое время — например показать уведомление через 3 секунды или запустить анимацию. JavaScript не ждёт эти 3 секунды стоя на месте — он передаёт задачу браузеру и продолжает работать. Когда время вышло — колбэк встаёт в очередь и Event Loop его подхватывает. Таймеры разберём подробно в следующей лекции.
- HTTP-запросы. Когда страница запрашивает данные с сервера — это может занять секунды. Благодаря Event Loop JavaScript не блокируется и не замораживает страницу. Браузер отправляет запрос, ждёт ответа сам, а когда данные пришли — колбэк встаёт в очередь. Подробно разберём в лекции про fetch.

- События пользователя. Каждый клик мышью, нажатие клавиши, прокрутка страницы — это события. Браузер следит за ними постоянно. Когда событие произошло — соответствующий обработчик встаёт в очередь задач, и Event Loop запускает его когда стек свободен.
- Промисы. Асинхронные операции, результат которых придёт позже — загрузка файла, запрос к базе данных. Когда результат готов — колбэк .then встаёт в очередь микрозадач. Промисы разберём в отдельной лекции.

Во всех этих сценариях одна и та же схема: JavaScript инициирует операцию → передаёт её браузеру → продолжает выполнять код → когда операция завершилась, колбэк встаёт в очередь → Event Loop запускает его когда стек пуст.

2. Очередь задач и как Event Loop с ней работает.

В предыдущем вопросе мы говорили об «очереди задач» в общем. На самом деле в браузере их три, и каждая работает по-своему:

- Очередь микрозадач — высокоприоритетные задачи: промисы, queueMicrotask. Выполняются сразу после текущего кода.
- Render Queue — очередь рендеринга. Браузер перерисовывает страницу примерно 60 раз в секунду. Только в браузере — в Node.js отсутствует.
- Очередь макрозадач — обычные задачи: таймеры, события, HTTP-запросы.

Правила Event Loop

Event Loop работает по строгим правилам в строгом порядке:

- Выполнить весь синхронный код в Call Stack до конца
- Выполнить все задачи из очереди микрозадач — до последней
- Выполнить рендеринг — если браузер решил что пришло время
- Взять одну задачу из очереди макрозадач и выполнить её
- Снова выполнить все микрозадачи которые появились
- Снова рендеринг если нужен
- Повторить с шага 4

Три ключевых момента: микрозадачи вычищаются полностью за один раз, рендеринг происходит между микрозадачами и макрозадачами, макрозадачи берутся по одной за цикл.

Разберем пример:

```
console.log("1");

queueMicrotask(() => console.log("2"));

queueMicrotask(() => console.log("3"));

console.log("4");
```

Разберём пошагово:

- Шаг 1: console.log("1") — синхронно, выполняется сразу. Вывод: 1
- Шаг 2: queueMicrotask(...) — колбэк с "2" встаёт в очередь микрозадач. В стек не идёт.
- Шаг 3: queueMicrotask(...) — колбэк с "3" встаёт в очередь микрозадач. В стек не идёт.
- Шаг 4: console.log("4") — синхронно, выполняется сразу. Вывод: 4
- Шаг 5: Стек пуст. Event Loop проверяет очередь микрозадач — там два колбэка. Выполняет оба по порядку. Вывод: 2, 3
- Шаг 6: Микрозадачи закончились. Браузер проверяет — нужен ли рендеринг. Если да — перерисовывает страницу.

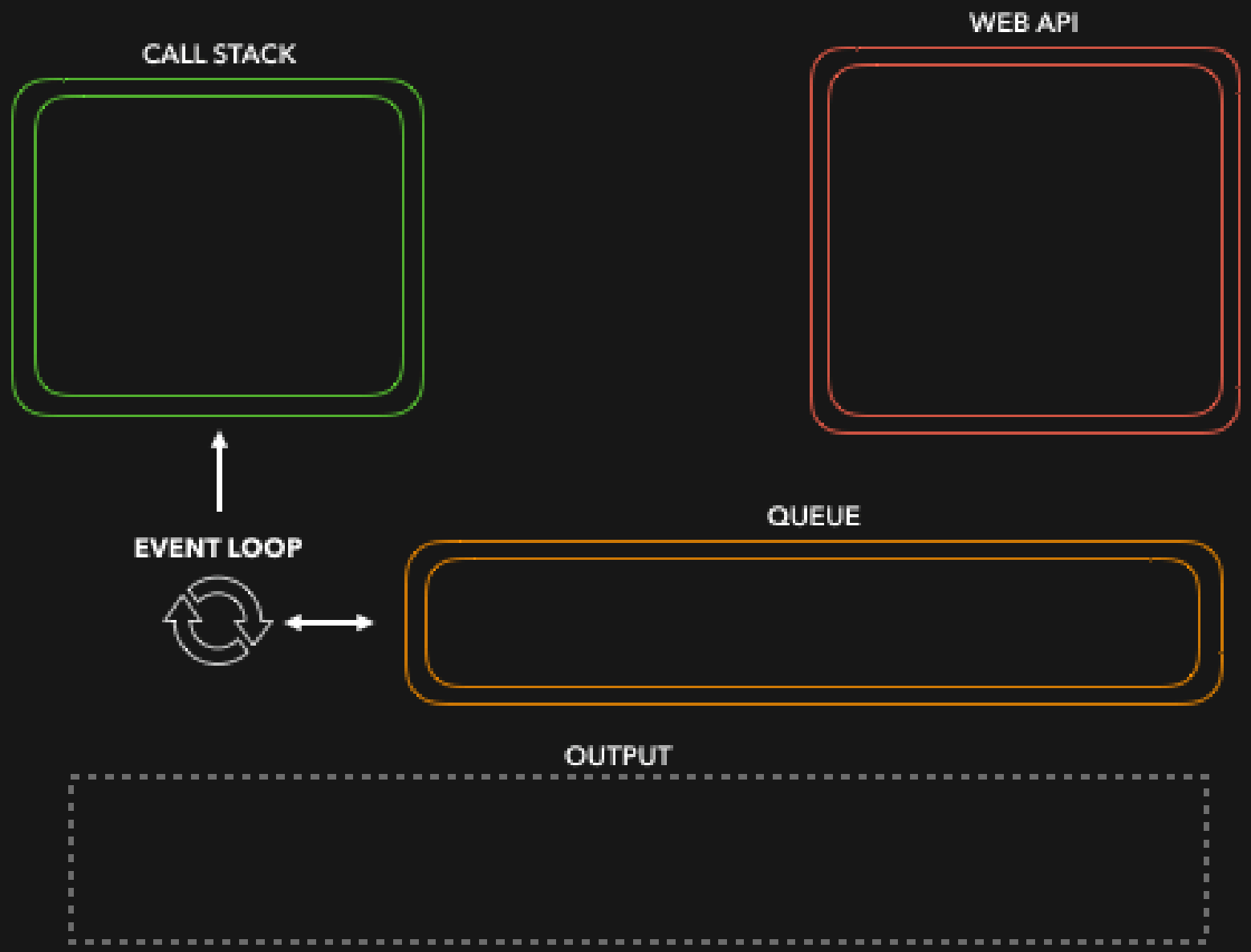
Итоговый вывод:

```
1
4
2
3
PS C:\Users\user\Documents\oop>
```

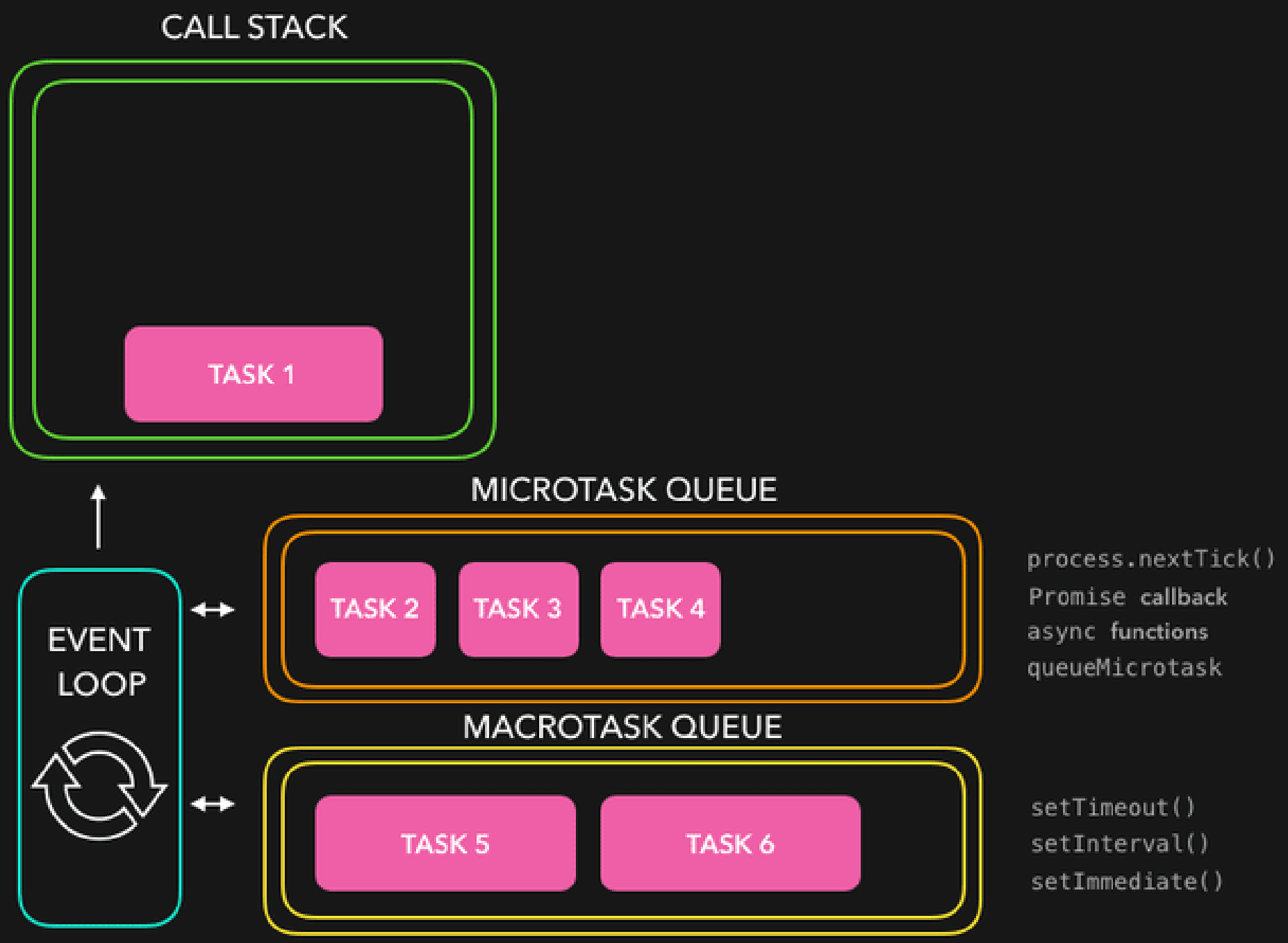
Render Queue — очередь рендеринга

Браузер старается перерисовывать страницу примерно 60 раз в секунду — то есть каждые примерно 16 мс. После того как все микрозадачи выполнены — браузер проверяет: пришло ли время для очередного рендеринга? Если да — перерисовывает страницу, обновляет DOM, применяет CSS. Только после этого переходит к макрозадачам.

Это объясняет важное практическое правило: если синхронный код выполняется слишком долго — страница зависает. Браузер не может добраться до Render Queue пока стек не пуст. Поэтому тяжёлые вычисления блокируют не только другой код, но и отрисовку интерфейса — пользователь видит замёрзшую страницу.



Made with ❤ by **Lydia Hallie**


CALL STACK
TASK 1
MICROTASK QUEUE
TASK 2
TASK 3
TASK 4
process.nextTick()
Promise callback
async functions
queueMicrotask
EVENT LOOP
MACROTASK QUEUE
TASK 5
TASK 6
setTimeout()
setInterval()
setImmediate()

Пример:

```
document.addEventListener('click', () => {
  // 1. Даем команду сменить цвет
  document.body.style.backgroundColor = 'red';

  // 2. Запускаем тяжелый цикл на 5 секунд
  const start = Date.now();
  while (Date.now() - start < 5000) {}

  console.log("Цикл окончен");
});
```

«Рендеринг» — это отдельный этап, который вполнится после того, как Call Stack полностью опустеет.

Итог:

- Очередей три: микрозадачи, Render Queue, макрозадачи
- Порядок за один цикл: синхронный код → все микрозадачи → рендеринг → одна макрозадача
- Микрозадачи вычищаются полностью — включая те, что добавляются внутри других микрозадач
- Render Queue — браузер перерисовывает страницу ~60 раз в секунду после микрозадач
- Долгий синхронный код блокирует и рендеринг — страница зависает
- Макрозадачи берутся по одной за цикл — таймеры, события, запросы разберём далее
- В Node.js Render Queue отсутствует

3. Macrotasks — что это и примеры.

Макрозадача — это единица работы, которую Event Loop берёт из очереди макрозадач и выполняет за один цикл. После каждой макрозадачи Event Loop проверяет очередь микрозадач и при необходимости выполняет рендеринг — только потом берёт следующую макрозадачу.

Если микрозадачи — это продолжение текущей операции, то макрозадачи — это новые независимые операции. Каждая макрозадача — отдельный «кусок работы», который пришёл извне.

Источники макрозадач

Источник	Когда создаётся макрозадача
setTimeout	когда истекает указанная задержка
setInterval	при каждом срабатывании интервала
События DOM	клик, нажатие клавиши, прокрутка
HTTP-запросы	когда пришёл ответ от сервера
MessageChannel	при отправке сообщения между потоками
setImmediate	в Node.js — после текущего цикла

Таймеры и HTTP-запросы ещё не изучались — разберём их в следующих лекциях. Здесь важно понять общий принцип: каждый из этих источников создаёт задачу, которая встаёт в очередь и ждёт своей очереди.

Ключевые особенности макрозадач:

- Берутся по одной. За один цикл Event Loop выполняет только одну макрозадачу. После неё — проверка микрозадач, рендеринг, и только потом следующая макрозадача. Это принципиальное отличие от микрозадач, которые вычищаются все разом.
- Выполняются в порядке очереди. Задачи с одинаковым приоритетом выполняются в том порядке, в котором попали в очередь — первым пришёл, первым обслужен.
- Не прерываются. Пока макрозадача выполняется — ничто другое не может вмешаться. JavaScript однопоточный, и текущая задача всегда доводится до конца.

Пример с событием — макрозадача без таймера

Событие клика — хороший пример макрозадачи, который не требует таймеров:

```
console.log("старт");

document.addEventListener("click", () => {
    console.log("клик — макрозадача");
});

console.log("конец");
```

При запуске выведется:

- старт
- конец

Обработчик клика зарегистрирован, но не выполнился — он ждёт в очереди макрозадач. Как только пользователь кликнет — браузер положит колбэк в очередь, и Event Loop выполнит его когда стек будет пуст.

Макрозадача vs микрозадача — разница в приоритете

```
console.log("синхронно");

queueMicrotask(() => console.log("микрозадача"));

document.addEventListener("click", () => console.log("макрозадача"));

console.log("тоже синхронно");
```

Если сразу кликнуть на страницу — вывод будет:

- синхронно
- тоже синхронно
- микрозадача
- макрозадача

Микрозадача выполнилась раньше макрозадачи — даже если событие клика произошло до того как JavaScript дошёл до queueMicrotask. Очередь микрозадач всегда обрабатывается перед макрозадачами.

Итог:

- Макрозадача — единица работы, которая берётся из очереди по одной за цикл
- Источники макрозадач: таймеры, события DOM, HTTP-запросы и другие Web API
- Каждая макрозадача выполняется полностью без прерываний
- После каждой макрозадачи — проверка микрозадач и рендеринг
- Макрозадачи выполняются в порядке очереди
- Выполнение по одной защищает интерфейс от зависания
- Микрозадачи всегда приоритетнее макрозадач

4. Microtasks — что это и примеры.

Микрозадача — это небольшая высокоприоритетная задача, которая выполняется сразу после текущего синхронного кода — до рендеринга и до любой макрозадачи.

Если макрозадачи — это новые независимые операции, которые пришли извне, то микрозадачи — это продолжение текущей операции. Они нужны когда результат должен быть обработан как можно скорее, без лишних задержек.

Источники микрозадач

Источник	Когда создаётся макрозадача
Promise.then / .catch / .finally	когда промис выполнился
queueMicrotask()	немедленно при вызове
MutationObserver	когда изменился DOM
async/await	под капотом использует промисы

Ключевые особенности микрозадач:

- Вычищаются все разом. В отличие от макрозадач — Event Loop не берёт микрозадачи по одной. Он выполняет их все подряд до последней, прежде чем двигаться дальше.
- Выполняются до рендеринга. Браузер не перерисует страницу пока очередь микрозадач не опустеет. Это гарантирует что пользователь увидит уже обновлённый результат.
- Могут добавлять новые микрозадачи. Если микрозадача создаёт новую микрозадачу — та тоже выполнится в этом же цикле. Event Loop не уйдёт дальше пока очередь не опустеет полностью.

Осторожно с бесконечным циклом. Если микрозадачи бесконечно добавляют друг друга — рендеринг и макрозадачи никогда не выполнятся. Страница зависнет:

```
// ❌ бесконечный цикл микрозадач — страница зависнет
function infinite() {
    queueMicrotask(infinite);
}
infinite();
```

Базовый пример:

```
console.log("1");

queueMicrotask(() => console.log("2"));

console.log("3");
```

```
PS C:\Users\user\Documents\oop> node index.js
1
3
2
```

queueMicrotask зарегистрировал колбэк в очереди микрозадач. JavaScript продолжил синхронный код, вывел "3". Только после того как стек опустел — Event Loop выполнил микрозадачу.

Несколько микрозадач — все выполняются разом:

```
console.log("старт");

queueMicrotask(() => console.log("микрозадача 1"));
queueMicrotask(() => console.log("микрозадача 2"));
queueMicrotask(() => console.log("микрозадача 3"));
```

```
старт
конец
микрозадача 1
микрозадача 2
микрозадача 3
```

Все три микрозадачи выполнились подряд до того как Event Loop перешёл дальше. Никакой макрозадачи и никакого рендеринга между ними не было.

Микрозадача внутри макрозадачи

Когда макрозадача создаёт микрозадачу — та выполняется сразу после этой макрозадачи, до следующей макрозадачи:

```
// имитируем две макрозадачи через события
document.addEventListener("click", () => {
    console.log("макрозадача 1");
    queueMicrotask(() => console.log("микрозадача внутри макрозадачи 1"));
});

document.addEventListener("click", () => {
    console.log("макрозадача 2");
});
```

При клике вывод будет:

```
макрозадача 1
микрозадача внутри макрозадачи 1
макрозадача 2
```

После выполнения первой макрозадачи Event Loop проверил очередь микрозадач — нашёл там микрозадачу и выполнил её. Только потом взял вторую макрозадачу.

Сравнение микрозадач и макрозадач

	Микрозадачи	**Макрозадачи**
Приоритет	Высокий	Обычный
Сколько за цикл	Все до последней	Одна
До рендеринга	Да	Нет
Источники	queueMicrotask, промисы	Таймеры, события, запросы
Добавление новых	Выполнятся в этом же цикле	Встанут в конец очереди

Практический смысл микрозадач

Микрозадачи нужны когда важно обработать результат до того как браузер что-то нарисует или сделает. Например, промис получил данные от сервера — их нужно обработать и записать в переменные прежде чем браузер перерисует страницу. Иначе пользователь может увидеть промежуточное некорректное состояние.

Именно поэтому промисы используют очередь микрозадач, а не макрозадач — это осознанное решение авторов JavaScript.

Итог:

- Микрозадача — высокоприоритетная задача, выполняется сразу после синхронного кода
- Все микрозадачи вычищаются до рендеринга и до следующей макрозадачи
- Микрозадача может добавить новую — она тоже выполнится в этом цикле
- Бесконечные микрозадачи заблокируют рендеринг и макрозадачи
- queueMicrotask() — способ добавить микрозадачу вручную
- Промисы и async/await тоже используют очередь микрозадач — разберём в следующих лекциях
- Микрозадачи нужны когда результат важно обработать до следующего рендеринга

4. queueMicrotask — добавление микрозадачи вручную.

queueMicrotask — это встроенная функция JavaScript, которая позволяет добавить колбэк в очередь микрозадач вручную.

Колбэк выполнится асинхронно — но раньше любой макрозадачи и раньше рендеринга.

Синтаксис:

queueMicrotask(функция);

Принимает ровно один аргумент — функцию без параметров. Передать аргументы в колбэк напрямую нельзя — используется замыкание:

```
// ❌ так нельзя — queueMicrotask не передаёт аргументы
queueMicrotask(console.log, "привет");

// ✅ через стрелочную функцию
queueMicrotask(() => console.log("привет"));

// ✅ через замыкание
const message = "привет";
queueMicrotask(() => console.log(message));
```

Где доступен queueMicrotask?

queueMicrotask доступен везде без импорта:

- Браузер — глобальная функция window.queueMicrotask, можно писать просто queueMicrotask
- Node.js — доступен с версии 11.0 как глобальная функция
- Не поддерживается в очень старых браузерах — но это уже не актуально

Когда использовать queueMicrotask?

queueMicrotask нужен когда важно выполнить код асинхронно — но с гарантией что это произойдёт до рендеринга и до любой макрозадачи.

Типичный сценарий — отложить работу до конца текущего кода, но не терять приоритет:

```
function processData() {
    const result = 42; // данные готовы

    // нужно сообщить о результате —
    // но после того как текущая функция завершится
    queueMicrotask(() => {
        console.log("данные обработаны:", result);
    });

    console.log("функция ещё выполняется...");
}

processData();
console.log("вызов завершён");
```

Вывод:

```
функция ещё выполняется...
вызов завершён
данные обработаны: 42
```

Разберём пошагово:

- Шаг 1: вызывается processData(), функция входит в стек.
- Шаг 2: result = 42 — данные готовы.
- Шаг 3: queueMicrotask(...) — колбэк встаёт в очередь микрозадач. Функция не останавливается и идёт дальше.
- Шаг 4: console.log("функция ещё выполняется...") — выводится синхронно.
- Шаг 5: processData() завершается, выходит из стека.
- Шаг 6: console.log("вызов завершён") — выводится синхронно.
- Шаг 7: Стек пуст. Event Loop забирает микрозадачу из очереди — выводится "данные обработаны: 42".

Ключевой момент — колбэк дождался пока весь текущий код завершился, и только потом выполнился. При этом он выполнился до рендеринга и до любого таймера — приоритет сохранился.

Итог:

- queueMicrotask(fn) — добавляет колбэк в очередь микрозадач вручную
- Колбэк выполнится после текущего синхронного кода — до рендеринга и до макрозадач
- Микрозадачи выполняются в порядке добавления — FIFO
- Новые микрозадачи добавленные внутри выполняются в этом же цикле
- Отличается от setTimeout(fn, 0) — микрозадача приоритетнее и выполняется до рендеринга
- Доступен глобально в браузере и Node.js (с версии 11)
- Бесконечные микрозадачи заблокируют рендеринг и макрозадачи

Итоги лекции:

JavaScript однопоточный — в один момент выполняется только одна задача

Event Loop — диспетчер, который решает что выполнять следующим

Участников четыре: Call Stack, Web API / Node API, очереди задач, Event Loop

Web API берёт долгие операции на себя — JavaScript не ждёт и идёт дальше

Очередей три: микрозадачи, Render Queue, макрозадачи

Порядок за один цикл: синхронный код → все микрозадачи → рендеринг → одна макрозадача

Микрозадачи вычищаются все до последней — включая добавленные внутри других микрозадач

Макрозадачи берутся по одной за цикл — таймеры, события DOM, HTTP-запросы

Render Queue — браузер перерисовывает страницу ~60 раз в секунду, после микрозадач

queueMicrotask(fn) — добавляет микрозадачу вручную, выполнится до рендеринга и макрозадач

В Node.js нет Render Queue — только микрозадачи и макрозадачи

Контрольне Вопросы:

- Что такое Event Loop — объясни своими словами.
- В каком порядке выполняются синхронный код, микрозадачи и макрозадачи?
- Чем микрозадача отличается от макрозадачи?
- Сколько микрозадач выполняется за один цикл Event Loop? А макрозадач?
- Что такое Render Queue и когда браузер выполняет рендеринг?
- Что произойдёт если синхронный код выполняется очень долго?
- Для чего используется queueMicrotask?

Домашнее задание:

https://ru.hexlet.io/programs/js-async

Материалы лекций:

https://github.com/ShViktor72/Education2025

Обратная связь:

colledge20education23@gmail.com